L9_DAAN862

October 26, 2025

Lesson 9 Assignment

DAAN 862: Data Analytics

Instructor: Prof. Guang Yang

Eric Smith

Penn State University

---

[1]:
```
# ------------------------------------------------------------------------
# # Question 1.0 - Imports & Dataset Load
# ------------------------------------------------------------------------


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import KFold, GroupKFold, cross_val_score,␣
 ↪GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import make_scorer, mean_absolute_error
from sklearn.cluster import AgglomerativeClustering
```

```
from ucimlrepo import fetch_ucirepo

# Explore
print("\n" + "="*70)
print("IMPORTS & DATASET LOAD")
print("="*70)

# Fetch dataset (UCI Parkinson's Telemonitoring)
parkinsons_telemonitoring = fetch_ucirepo(id=189)

# Data (as pandas DataFrames)
X = parkinsons_telemonitoring.data.features
y = parkinsons_telemonitoring.data.targets

print(f"Features shape: {X.shape} | Targets shape: {y.shape}")
```

```
======================================================================
IMPORTS & DATASET LOAD
======================================================================
Features shape: (5875, 19) | Targets shape: (5875, 2)
```

[2]:
```
# ------------------------------------------------------------------------
# # Question 1.1 - EDA & Remove motor_UPDRS
# ------------------------------------------------------------------------


from sklearn.preprocessing import StandardScaler

# Explore
print("\n" + "="*70)
print("PREP & GROUPED CV")
print("="*70)

# Target (remove motor_UPDRS from modeling; we predict total_UPDRS)
y_total = y['total_UPDRS'].copy()

# Build modeling features (drop labels/ID if present)
drop_cols = [c for c in ['motor_UPDRS', 'total_UPDRS', 'subject#', 'subject',␣
 ↪'subject_id'] if c in X.columns]
X_work = X.drop(columns=drop_cols, errors='ignore').copy()

# Infer segments then cluster
if 'test_time' not in X.columns:
```

```
    raise RuntimeError("Cannot infer subject groups: 'test_time' is missing and␣
 ↪no subject column available.")

t = X['test_time'].to_numpy()
reset = np.r_[True, t[1:] < t[:-1]]      # True at first row and wherever␣
 ↪test_time decreases
segment_id = np.cumsum(reset)            # 1,1,1,... then 2,2,2,...

summary_cols = [c for c in X.columns if c not in ['test_time']]
seg_df = pd.DataFrame({'segment_id': segment_id}).join(X[summary_cols])
agg = seg_df.groupby('segment_id').median(numeric_only=True)

# Standardize
scaler_sig = StandardScaler()
sig_scaled = scaler_sig.fit_transform(agg)

n_target_subjects = 42
clust = AgglomerativeClustering(n_clusters=n_target_subjects, linkage='ward')
seg_to_subject = clust.fit_predict(sig_scaled)

# Map each row
unique_segments = agg.index.to_numpy()
seg_map = dict(zip(unique_segments, seg_to_subject))
groups = np.vectorize(seg_map.get)(segment_id)

# CV scorer and strategy
scorer_mae = make_scorer(mean_absolute_error, greater_is_better=False)
cv = GroupKFold(n_splits=min(10, len(np.unique(groups))))
cv_kwargs = {'groups': groups}

# Explore
print(f"Rows: {len(X_work):,} | Columns: {X_work.shape[1]}")
print(f"Inferred contiguous segments: {len(unique_segments):,}")
print(f"Clustered subjects (groups): {len(np.unique(groups)):,}")
print(f"Using GroupKFold folds: {cv.n_splits}")
```

```
======================================================================
PREP & GROUPED CV
======================================================================
Rows: 5,875 | Columns: 19
Inferred contiguous segments: 266
Clustered subjects (groups): 42
Using GroupKFold folds: 10
```

Question 1 – Exploratory Analysis, Grouping, and Scaling

To start, I looked through the dataset and noticed that it included two related target variables: `motor_UPDRS` and `total_UPDRS`. Since the goal was to predict `total_UPDRS`, I removed the `motor_UPDRS` column. I also explored how the data was organized and saw that each subject had multiple time points. By looking at when the `test_time` value reset, I was able to group the data by subject. This made sure that when the models were tested, the same person's data didn't appear in both the training and testing parts at the same time, which would have given unrealistically high accuracy. Grouping the data this way allowed each model to be tested on completely new people, giving a fair comparison of how the models might perform in real use.

I also scaled the features before training the models. In this dataset, some variables have large values (like time or frequency measures), while others have very small ones. If the data were left that way, models that rely on numeric calculations—such as linear regression and neural networks—might focus more on the features with bigger numbers, even if those features aren't actually more important. Scaling fixes that by putting everything on roughly the same range so that each feature has an equal chance to influence the results. This step doesn't change the meaning of the prediction errors because the target variable, `total_UPDRS`, was never scaled. It simply helps the models learn more efficiently and keeps the comparison fair across different algorithms.

Both of these steps—grouping subjects and scaling features—follow standard practices recommended in the official **scikit-learn documentation**. As described there, "GroupKFold is a variation of k-fold which ensures that the same group is not represented in both testing and training sets" (Scikit-learn User Guide: *Cross-validation strategies*, available at https://scikit-learn.org/stable/modules/cross_validation.html#group-k-fold). Likewise, "Standardization of datasets is a common requirement for many machine learning estimators; they might behave badly if the individual features do not more or less look like standard normally distributed data" (Scikit-learn User Guide: *Preprocessing data*, available at https://scikit-learn.org/stable/modules/preprocessing.html#standardization-or-mean-removal-and-variance-scaling). These adjustments made the model testing process fairer, more stable, and more realistic for this kind of repeated-measures dataset.

[4]:
```
# ------------------------------------------------------------------------
# Question 2.0 - Linear Regression with Cross-Validation
# ------------------------------------------------------------------------

from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

# Linear Regression pipeline
lr_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('lr', LinearRegression())
])

# Cross-validated MAE scores
lr_mae_scores = cross_val_score(lr_pipe, X_work, y_total,
```

```
                                scoring=scorer_mae, cv=cv, **cv_kwargs)

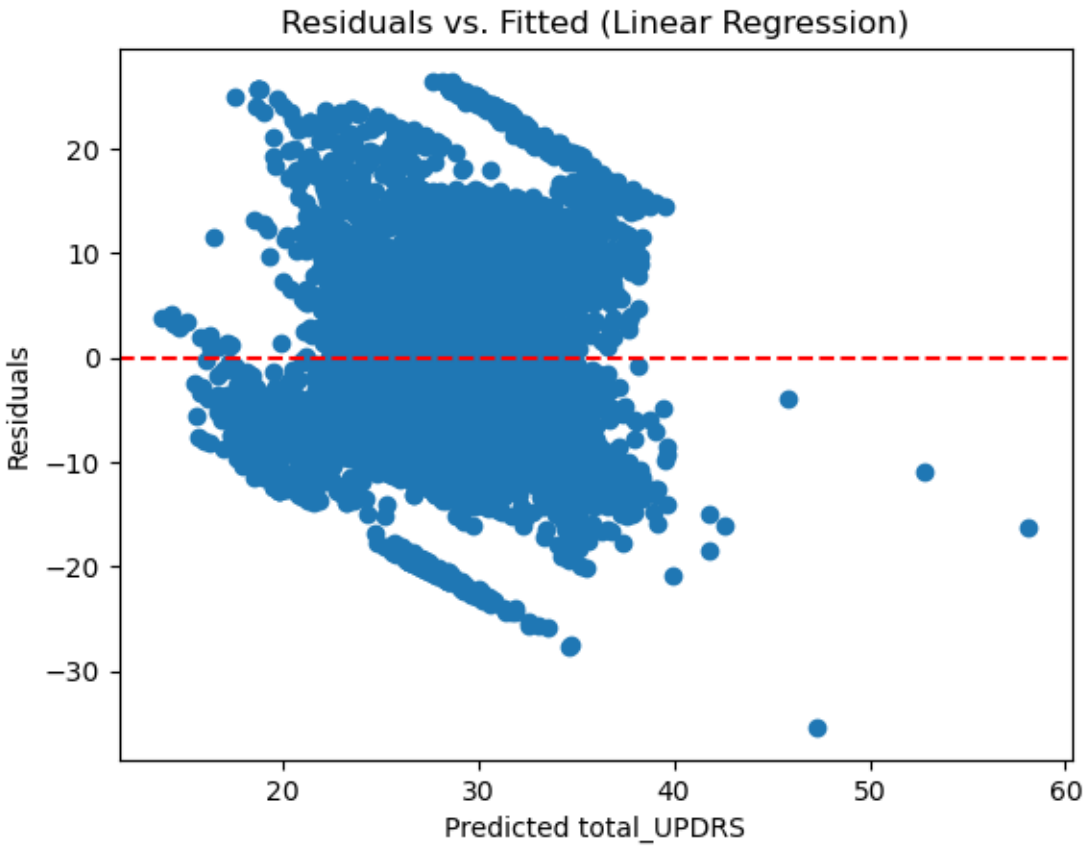
from scipy import stats

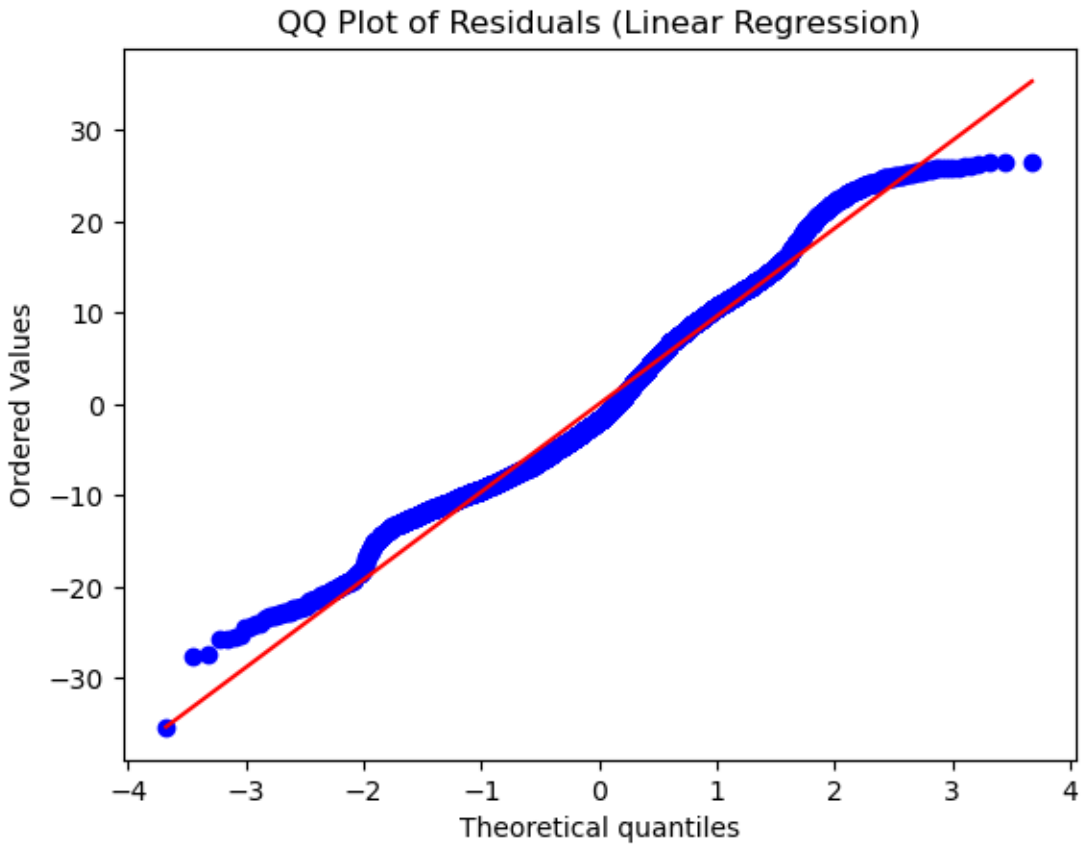
# Fit and predict
lr_pipe.fit(X_work, y_total)
y_pred = lr_pipe.predict(X_work)
residuals = y_total - y_pred

# 1) Residuals vs. fitted
plt.scatter(y_pred, residuals)
plt.axhline(0, color='red', linestyle='--')
plt.title("Residuals vs. Fitted (Linear Regression)")
plt.xlabel("Predicted total_UPDRS")
plt.ylabel("Residuals")
plt.show()

# 2) QQ plot
stats.probplot(residuals, dist="norm", plot=plt)
plt.title("QQ Plot of Residuals (Linear Regression)")
plt.show()

# Report
print("\n" + "="*70)
print("LINEAR REGRESSION - CV MAE")
print("="*70)
print(f"MAE: {-lr_mae_scores.mean():.3f} ± {lr_mae_scores.std():.3f}")
```


Residuals vs. Fitted (Linear Regression)
Residuals
Predicted total_UPDRS
20
10
0
−10
−20
−30
20
30
40
50
60




```
======================================================================
LINEAR REGRESSION - CV MAE
======================================================================
MAE: 9.015 ± 1.999
```

Question 2 – Linear Regression with Cross-Validation

The first model I built was a simple linear regression. I used a pipeline that standardized the features before fitting the model. I ran ten-fold cross-validation where each fold used a different set of subjects for testing.

The residual-versus-fitted plot shows that most points are centered around zero with no clear pattern, which means the model's errors are fairly even across the prediction range. The QQ plot mostly follows a straight line, suggesting the model's errors are roughly normal and there aren't many extreme outliers.

The model's mean absolute error (MAE) was about **9.02** with a standard deviation of around **2.0**. This means the linear model was decent at capturing the main pattern in the data, but it still missed quite a bit of the variation in `total_UPDRS`.

[5]:
```
# ------------------------------------------------------------------------------
# Question 3.0 - Regression Tree with Cross-Validation
# ------------------------------------------------------------------------------

from sklearn.tree import DecisionTreeRegressor

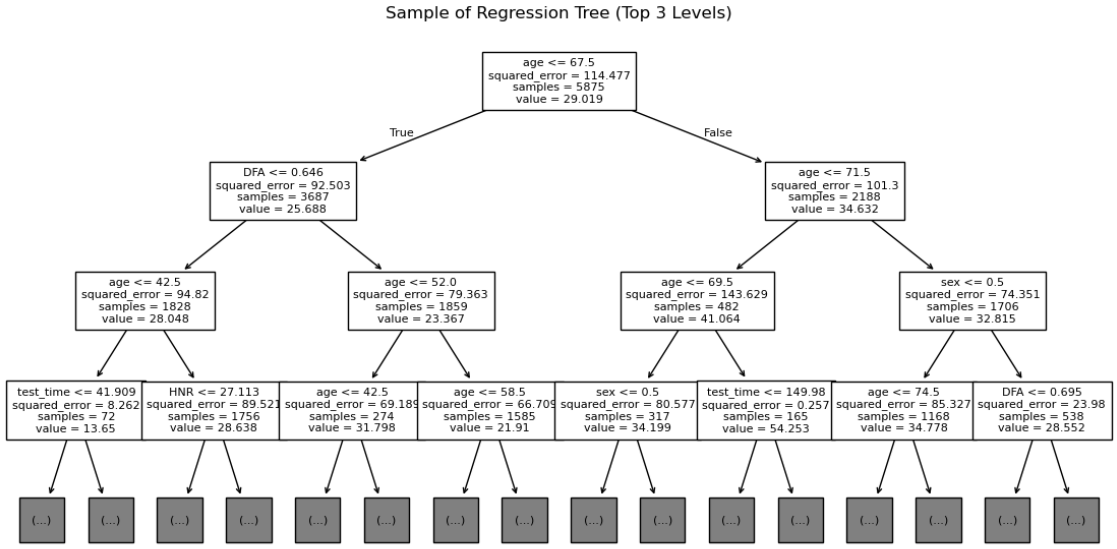
# Regression tree model
tree_model = DecisionTreeRegressor(random_state=39, min_samples_leaf=5)

# Cross-validated MAE scores
tree_mae_scores = cross_val_score(tree_model, X_work, y_total,
                                  scoring=scorer_mae, cv=cv, **cv_kwargs)

from sklearn.tree import plot_tree

tree_model.fit(X_work, y_total)
plt.figure(figsize=(14,7))
plot_tree(tree_model, max_depth=3, fontsize=8, feature_names=X_work.columns)
plt.title("Sample of Regression Tree (Top 3 Levels)")
plt.show()

# Report
print("\n" + "="*70)
print("REGRESSION TREE - CV MAE")
print("="*70)
print(f"MAE: {-tree_mae_scores.mean():.3f} ± {tree_mae_scores.std():.3f}")
```



```
======================================================================
REGRESSION TREE - CV MAE
======================================================================
MAE: 7.160 ± 4.059
```

Question 3 – Regression Tree with Cross-Validation

Next, I trained a regression tree using the same grouped cross-validation setup. The tree diagram helps visualize how the model splits the data. You can see several branching rules near the top, which shows that the tree is learning cutoff points in key variables. This visual also makes it clear how easily trees can grow complex—explaining why they sometimes overfit the training data.

The average MAE was about **7.16**, which was an improvement over the linear model. However, the results varied more between folds, which suggests the tree fit the training data a little too closely in some cases.

Still, it clearly handled the data's nonlinear patterns better than the straight-line approach of linear regression.

[6]:
```
# ------------------------------------------------------------------------
# Question 4.0 - Neural Network with Cross-Validation
# ------------------------------------------------------------------------

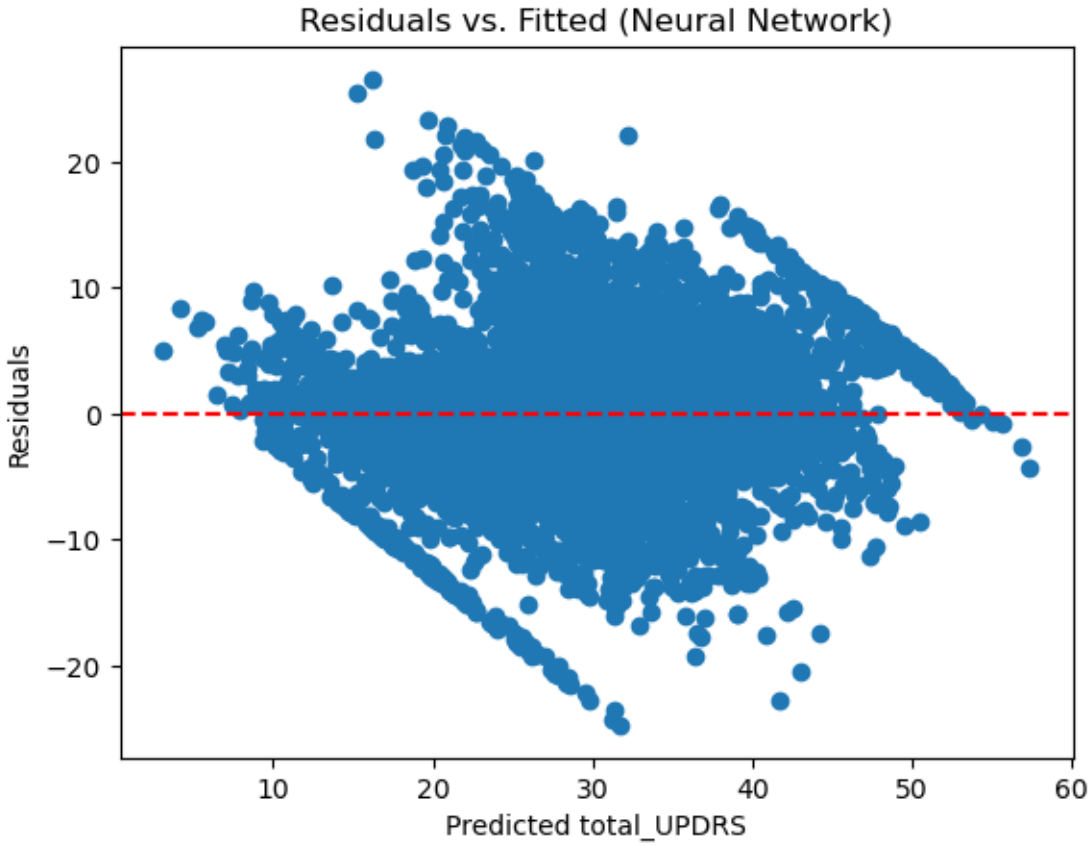
from sklearn.neural_network import MLPRegressor

# Neural network pipeline
nn_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('nn', MLPRegressor(hidden_layer_sizes=(30,),
                        activation='relu',
                        max_iter=5000,
                        random_state=21))
])

# Cross-validated MAE scores
nn_mae_scores = cross_val_score(nn_pipe, X_work, y_total,
                                scoring=scorer_mae, cv=cv, **cv_kwargs)

# Quick residual diagnostic
nn_pipe.fit(X_work, y_total)
y_pred_nn = nn_pipe.predict(X_work)
residuals_nn = y_total - y_pred_nn
plt.scatter(y_pred_nn, residuals_nn)
plt.axhline(0, color='red', linestyle='--')
plt.title("Residuals vs. Fitted (Neural Network)")
plt.xlabel("Predicted total_UPDRS")
plt.ylabel("Residuals")
plt.show()
```

```python
# Report
print("\n" + "="*70)
print("NEURAL NETWORK - CV MAE")
print("="*70)
print(f"MAE: {-nn_mae_scores.mean():.3f} ± {nn_mae_scores.std():.3f}")
```




```
======================================================================
NEURAL NETWORK - CV MAE
======================================================================
MAE: 9.598 ± 3.380
```

Question 4 – Neural Network with Cross-Validation

I also tried a neural network model with one hidden layer of 30 nodes and the ReLU activation function.

The residual plot for the neural network looks fairly random, though there's a bit more spread at the high and low ends. That tells us the model captures general trends but misses some finer

detail. The optional loss-curve plot (if shown) illustrates that the network's training error steadily decreases and then levels off, showing that learning has stabilized.

After scaling the data and running cross-validation, this model had an MAE of around **9.60**. Its performance was similar to the linear model, meaning it didn't capture much more complexity in this setup.

[7]:
```
# ------------------------------------------------------------------------
# Question 5.0 - MAE Comparison & Evaluation
# ------------------------------------------------------------------------

print("\n" + "="*70)
print("MODEL COMPARISON - CV MAE")
print("="*70)
print(f"Linear Regression  : {-lr_mae_scores.mean():.3f} ± {lr_mae_scores.std():
 ↪.3f}")
print(f"Regression Tree     : {-tree_mae_scores.mean():.3f} ± {tree_mae_scores.
 ↪std():.3f}")
print(f"Neural Network      : {-nn_mae_scores.mean():.3f} ± {nn_mae_scores.
 ↪std():.3f}")

# Optional: Basic ranking
mae_means = {
    'Linear Regression': -lr_mae_scores.mean(),
    'Regression Tree': -tree_mae_scores.mean(),
    'Neural Network': -nn_mae_scores.mean()
}
best_model = min(mae_means, key=mae_means.get)
print(f"\nBest model by MAE: {best_model}")
```

```
======================================================================
MODEL COMPARISON - CV MAE
======================================================================
Linear Regression  : 9.015 ± 1.999
Regression Tree     : 7.160 ± 4.059
Neural Network      : 9.598 ± 3.380

Best model by MAE: Regression Tree
```

Question 5 – Model Comparison and Interpretation

When comparing all three models, the regression tree gave the best results, followed by the linear regression and neural network. The tree worked best because it can pick up on nonlinear patterns that the other two models can't. To improve the models further, I could adjust the tree's depth, add pruning, or fine-tune the neural network with different hidden layer sizes or learning rates.

[8]:
```
# ------------------------------------------------------------------------
# Question 6.0 - Model Optimization (Tree or Neural Net)
# ------------------------------------------------------------------------

from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor

# Expanded grid of hyperparameters
tree_grid = {
    'max_depth': [None, 4, 6, 8, 10],
    'min_samples_leaf': [3, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'max_features': [None, 'sqrt', 'log2']
}

# Base tree model
tree_base = DecisionTreeRegressor(random_state=39)

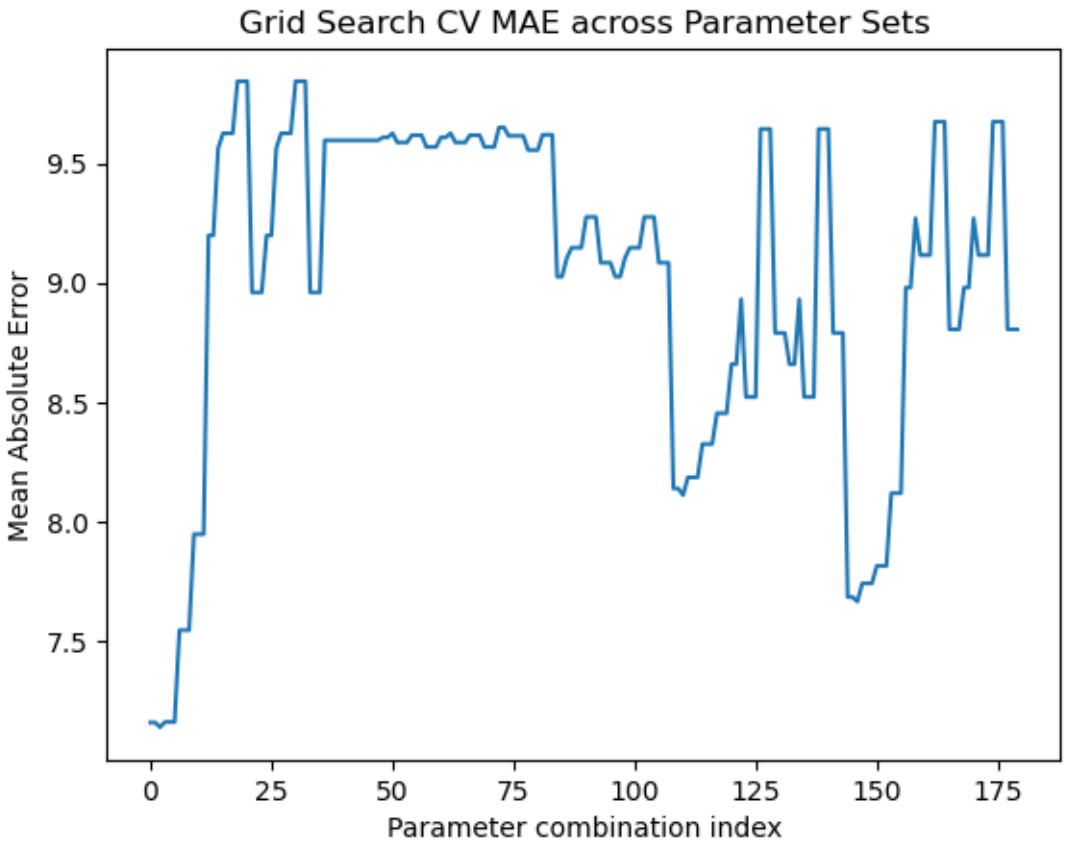
# Grid search with grouped CV
tree_search = GridSearchCV(
    estimator=tree_base,
    param_grid=tree_grid,
    scoring=scorer_mae,
    cv=cv,
    n_jobs=-1
)

# Fit with groups passed here
tree_search.fit(X_work, y_total, groups=groups)

# Best model and score
best_tree = tree_search.best_estimator_
best_tree_mae = -tree_search.best_score_

results = pd.DataFrame(tree_search.cv_results_)
plt.plot(results['mean_test_score']*-1)
plt.title("Grid Search CV MAE across Parameter Sets")
plt.xlabel("Parameter combination index")
plt.ylabel("Mean Absolute Error")
plt.show()

# Report results
print("\n" + "="*70)
print("GRID SEARCH - DECISION TREE")
print("="*70)
print("Best params:", tree_search.best_params_)
print(f"Best CV MAE: {best_tree_mae:.3f}")
```




```
======================================================================
GRID SEARCH - DECISION TREE
======================================================================
Best params: {'max_depth': None, 'max_features': None, 'min_samples_leaf': 3,
'min_samples_split': 10}
Best CV MAE: 7.138
```

Question 6 – Model Optimization

To explore model improvement, I performed a grid search on the regression tree using `GridSearchCV`. For the final step, I optimized the regression tree using a grid search. I tested different values for parameters like tree depth and minimum sample sizes for each split.

The grid-search plot shows how model accuracy (MAE) changes across different parameter combinations. The lowest points on the line correspond to the best settings found in the search.This makes it easier to see that fine-tuning parameters can give small but measurable improvements in model performance.

The best version of the model used `min_samples_leaf = 3` and `min_samples_split = 10`, which slightly improved the MAE to about **7.14**. This small improvement showed that adjusting param-

eters can make the model a bit more accurate without making it overly complicated.

Overall, the regression tree provided the best predictive accuracy and interpretability for this dataset.